

LAPORAN PROJECT
IMPLEMENTASI METODE PARALEL TOPSIS UNTUK SISTEM PENDUKUNG
KEPUTUSAN PEMILIHAN PRODUK TERBAIK PADA E-COMMERCE BERBASIS
MULTI-KRITERIA

Dosen Pengampu : Heri Wijayanto, S.T., M.T., Ph.D.



DISUSUN OLEH:

Inaz Putri Kamila	F1D02410056
Baiq Febylia Eka Ningrum	F1D02410109
Mustafida	F1D02410126
Istiqomah Virginia	F1D02410116

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MATARAM
2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah Swt., Tuhan Yang Maha Esa, atas limpahan rahmat dan hidayah-Nya sehingga laporan tugas besar ini dapat diselesaikan dengan baik. Laporan Project ini berjudul “Implementasi Paralel Metode TOPSIS untuk Sistem Pendukung Keputusan Pemilihan Produk Terbaik pada E-Commerce berbasis Multi-Kriteria”, yang disusun dalam rangka memenuhi tugas besar mata kuliah Pemrosesan Paralel Kelas C Tahun 2026. Penulis menyadari bahwa tanpa adanya bimbingan dan dukungan dari berbagai pihak, laporan ini tidak akan tersusun secara maksimal. Oleh karena itu, penulis mengucapkan terima kasih yang sebesar-besarnya kepada Bapak Heri Wijayanto S.T., M.T., Ph.D. selaku dosen pengampu mata kuliah, atas segala bimbingan, arahan, serta masukan yang sangat berarti selama proses penyusunan project ini.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh sebab itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan dan penyempurnaan di masa yang akan datang. Harapan penulis, melalui gagasan pengembangan project Implementasi Paralel Metode TOPSIS untuk Sistem Pendukung Keputusan Pemilihan Produk Terbaik pada E-Commerce berbasis Multi-Kriteria ini dapat menjadi solusi digital yang membantu sivitas akademika dan masyarakat luas dalam mengakses informasi terkait TOPSIS dengan cepat, akurat, dan efisien. Semoga laporan ini dapat memberikan manfaat bagi semua pihak yang membacanya, serta dapat menjadi kontribusi nyata dalam membangun kepedulian sosial melalui pemanfaatan teknologi informasi.

Mataram, 26 Mei 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang pesat dan tren digitalisasi yang meningkat telah membawa perubahan signifikan pada dunia bisnis, khususnya di sektor e-commerce. Saat ini, e-commerce merupakan sarana utama untuk melakukan transaksi jual beli, menawarkan kemudahan, efisiensi, dan jangkauan pasar yang luas bagi bisnis dan konsumen [1]. Dengan banyaknya platform e-commerce yang tersedia, seperti Shopee, Tokopedia, Lazada, dan lainnya, memilih platform yang tepat sangat penting. Setiap platform memiliki kelebihan dan kekurangannya masing-masing, termasuk harga, peringkat, layanan, jumlah pengguna, dan kemudahan penggunaan. Situasi ini membuat proses pengambilan keputusan menjadi lebih kompleks, terutama ketika banyak kriteria harus dipertimbangkan secara bersamaan.

Dalam situasi ini, pengambilan keputusan yang efektif sangat penting, terutama bagi pengguna dan bisnis yang ingin memilih platform e-commerce terbaik yang sesuai dengan kebutuhan dan tujuan mereka. Tanpa metode yang sistematis, proses pemilihan cenderung subjektif dan tidak akurat. Oleh karena itu, diperlukan pendekatan yang dapat membantu mengevaluasi berbagai alternatif secara objektif dan terstruktur.

Salah satu pendekatan yang dapat digunakan adalah Sistem Pendukung Keputusan (SPK) dengan metode Multi-Criteria Decision Making (MCDM). Metode ini memungkinkan pemrosesan simultan beberapa kriteria, sehingga menghasilkan keputusan yang lebih optimal. Dalam studi ini, metode yang digunakan adalah Teknik untuk Penentuan Urutan Preferensi Berdasarkan Kesamaan dengan Solusi Ideal (TOPSIS), yang menentukan alternatif terbaik berdasarkan kedekatan dengan solusi ideal positif dan jarak dari solusi ideal negatif [2].

Metode TOPSIS dipilih karena secara konseptual sederhana, mudah diimplementasikan, dan menghasilkan peringkat alternatif yang jelas [2]. Dengan menggunakan metode ini, berbagai platform e-commerce dapat dievaluasi dan dibandingkan berdasarkan kriteria yang telah ditentukan, sehingga menghasilkan rekomendasi objektif untuk platform terbaik. Namun, seiring bertambahnya jumlah data dan kompleksitas perhitungan, terutama ketika melibatkan banyak alternatif dan kriteria, proses komputasi TOPSIS dapat menjadi lebih lambat.

Oleh karena itu, diperlukan pendekatan tambahan untuk meningkatkan efisiensi sistem, yaitu implementasi komputasi paralel. Pemrosesan paralel memungkinkan

perhitungan dilakukan secara simultan, sehingga mempercepat waktu eksekusi dan meningkatkan kinerja sistem.

Berdasarkan permasalahan tersebut, penelitian ini mengusulkan implementasi metode TOPSIS berbasis multi-kriteria dengan dukungan komputasi paralel dalam Sistem Pendukung Keputusan. Diharapkan sistem ini dapat membantu pengguna dalam menentukan pilihan terbaik dengan cepat, akurat, dan efisien, serta berkontribusi pada pengembangan metode pengambilan keputusan di sektor e-commerce.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah yang bisa disusun adalah sebagai berikut:

1. Bagaimana cara menentukan platform e-commerce terbaik berdasarkan banyak kriteria seperti harga, peringkat, layanan, jumlah pengguna, dan kemudahan penggunaan?
2. Bagaimana mengatasi kompleksitas perhitungan dalam metode TOPSIS ketika jumlah alternatif dan kriteria semakin banyak?
3. Bagaimana implementasi komputasi paralel dapat meningkatkan efisiensi dan kecepatan proses perhitungan dalam metode TOPSIS?
4. Seberapa efektif sistem yang dibangun dalam memberikan rekomendasi platform e-commerce terbaik secara cepat, akurat, dan efisien?

1.3 Tujuan

Penelitian ini dilatarbelakangi oleh kebutuhan pengambilan keputusan multikriteria dalam memilih platform e-commerce terbaik yang semakin kompleks akibat banyaknya alternatif dan kriteria yang harus dipertimbangkan. Proses ini sulit dilakukan secara manual karena dapat menimbulkan subjektivitas dan ketidakakuratan. Oleh karena itu, tujuan dari penelitian ini adalah:

- a. Menerapkan metode *Technique for Order Preference by Similarity to Ideal Solution* (TOPSIS) dalam Sistem Pendukung Keputusan untuk membantu pemilihan produk e-commerce terbaik berdasarkan berbagai kriteria.
- b. Mengimplementasikan komputasi paralel untuk meningkatkan efisiensi dan kecepatan proses perhitungan metode TOPSIS, terutama pada data dengan jumlah alternatif dan kriteria yang besar.
- c. Mengevaluasi kinerja sistem dalam hal kecepatan dan efisiensi komputasi dengan membandingkan metode paralel dan metode serial.

- d. Menghasilkan rekomendasi platform e-commerce yang optimal berdasarkan kedekatan dengan solusi ideal positif dan jarak dari solusi ideal negatif secara objektif dan terstruktur.

1.4 Manfaat

Kegiatan ini diharapkan memberikan kontribusi baik secara teoritis maupun praktis dalam bidang komputasi paralel dan pengambilan keputusan berbasis multi-kriteria. Berdasarkan tujuan yang telah dirumuskan, terdapat sejumlah manfaat yang dapat diperoleh, antara lain:

- a. Memberikan referensi praktis bagi penerapan metode *Technique for Order Preference by Similarity to Ideal Solution* dan pemrograman paralel (MPI) dalam konteks pemilihan produk pada e-commerce, sehingga memperkaya wawasan tentang teknik komputasi paralel dan pengambilan keputusan berbasis *Multi-Criteria Decision Making* di kalangan akademis maupun praktisi.
- b. Meningkatkan efisiensi waktu pemrosesan data dalam proses pemilihan produk terbaik melalui pemrosesan paralel berbasis MPI, selaras dengan tujuan utama komputasi paralel yaitu mempercepat kinerja komputasi terutama pada data berskala besar.
- c. Membantu pengguna dalam menentukan produk terbaik di *e-commerce* secara lebih cepat, objektif, dan akurat berdasarkan berbagai kriteria seperti harga, rating, dan popularitas, karena metode TOPSIS mampu menghasilkan peringkat alternatif berdasarkan kedekatan terhadap solusi ideal.

BAB II

TINJAUAN PUSTAKA

2.1 Multi-Criteria Decision Making (MCDM)

2.2.1. Konsep Dasar MCDM

Multi-Criteria Decision Making (MCDM) merupakan suatu pendekatan dalam pengambilan keputusan yang digunakan untuk menentukan alternatif terbaik dari sejumlah pilihan berdasarkan berbagai kriteria yang telah ditentukan. Dalam praktiknya, pengambilan keputusan di dunia nyata jarang hanya melibatkan satu faktor saja, melainkan mencakup berbagai aspek seperti biaya, kualitas, efisiensi, dan risiko. Oleh karena itu, MCDM digunakan untuk membantu proses evaluasi alternatif secara sistematis dan terstruktur dengan mempertimbangkan seluruh kriteria yang relevan [3].

Pendekatan MCDM memungkinkan penggabungan antara kriteria yang bersifat kuantitatif dan kualitatif dalam satu kerangka analisis. Hal ini menjadikan metode ini fleksibel untuk diterapkan dalam berbagai bidang, seperti sistem pendukung keputusan, manajemen, serta teknologi informasi. Dalam prosesnya, MCDM biasanya melibatkan penyusunan matriks keputusan yang berisi nilai masing-masing alternatif terhadap setiap kriteria, yang kemudian diolah menggunakan metode tertentu untuk menghasilkan nilai preferensi atau peringkat alternatif [4].

Tujuan utama dari MCDM adalah membantu pengambil keputusan dalam memilih alternatif terbaik melalui proses evaluasi yang objektif dan dapat dipertanggungjawabkan. Dengan adanya pendekatan ini, keputusan yang dihasilkan tidak hanya berdasarkan intuisi, tetapi juga didukung oleh analisis yang logis dan terukur. Selain itu, MCDM juga meningkatkan transparansi dalam proses pengambilan keputusan karena setiap tahapan dapat dijelaskan secara sistematis [1].

Secara umum, MCDM dibagi menjadi dua kategori utama, yaitu Multi Attribute Decision Making (MADM) dan Multi Objective Decision Making (MODM). MADM digunakan untuk permasalahan dengan jumlah alternatif yang terbatas, sedangkan MODM digunakan untuk permasalahan yang bersifat kontinu dan melibatkan fungsi tujuan matematis. Kedua pendekatan ini memiliki karakteristik yang berbeda, namun sama-sama bertujuan untuk menghasilkan keputusan yang optimal berdasarkan kriteria yang telah ditentukan [1].

2.2.2. Alternatif, Kriteria, dan Bobot

Dalam metode MCDM, terdapat tiga komponen utama yang menjadi dasar dalam proses pengambilan keputusan, yaitu alternatif, kriteria, dan bobot. Alternatif merupakan

sekumpulan pilihan yang akan dievaluasi untuk menentukan solusi terbaik sesuai dengan tujuan yang ingin dicapai. Setiap alternatif memiliki karakteristik yang berbeda dan akan dibandingkan berdasarkan nilai yang dimilikinya terhadap masing-masing kriteria [1].

Kriteria merupakan faktor atau aspek yang digunakan sebagai dasar dalam menilai setiap alternatif. Kriteria berfungsi sebagai parameter dalam proses evaluasi, sehingga dapat menentukan tingkat kualitas atau kelayakan suatu alternatif. Dalam MCDM, kriteria umumnya dibedakan menjadi dua jenis, yaitu kriteria benefit dan kriteria cost. Kriteria benefit adalah kriteria yang mengutamakan nilai terbesar sebagai nilai terbaik, seperti kualitas dan performa, sedangkan kriteria cost adalah kriteria yang mengutamakan nilai terkecil sebagai nilai terbaik, seperti harga dan waktu pengerjaan [4].

Selain itu, setiap kriteria memiliki bobot yang menunjukkan tingkat kepentingannya dalam proses pengambilan keputusan. Bobot ini mencerminkan seberapa besar pengaruh suatu kriteria terhadap hasil akhir keputusan. Dalam praktiknya, bobot biasanya dinormalisasi sehingga total keseluruhannya bernilai satu untuk mempermudah proses perhitungan dan menjaga konsistensi hasil evaluasi. Penentuan bobot dapat dilakukan berdasarkan preferensi pengambil keputusan atau menggunakan metode tertentu seperti Analytical Hierarchy Process (AHP) [1].

Melalui kombinasi antara alternatif, kriteria, dan bobot, MCDM menghasilkan nilai preferensi yang digunakan untuk menentukan peringkat dari setiap alternatif. Alternatif dengan nilai preferensi tertinggi akan dipilih sebagai solusi terbaik. Dengan demikian, penggunaan ketiga komponen tersebut memungkinkan proses pengambilan keputusan dilakukan secara lebih objektif, sistematis, dan dapat dipertanggungjawabkan [1].

2.2 Technique for Order Preference by Similarity to Ideal Solution (TOPSIS)

Metode *Technique for Order Preference by Similarity to Ideal Solution* (TOPSIS) merupakan salah satu metode pengambilan keputusan multikriteria (*Multi Criteria Decision Making* / MCDM) yang didasarkan pada konsep bahwa alternatif terbaik adalah alternatif yang memiliki jarak paling dekat dengan solusi ideal positif dan jarak paling jauh dari solusi ideal negatif [2].

TOPSIS menggunakan pendekatan geometris dengan memanfaatkan jarak Euclidean untuk mengukur kedekatan suatu alternatif terhadap solusi ideal. Dengan demikian, setiap alternatif akan dievaluasi berdasarkan dua aspek utama, yaitu kedekatannya terhadap kondisi terbaik dan jauhnya dari kondisi terburuk. Selain itu, metode TOPSIS mempertimbangkan seluruh kriteria secara simultan, sehingga mampu memberikan hasil

keputusan yang lebih objektif dan terstruktur dalam berbagai permasalahan pengambilan keputusan yang kompleks [5].

Metode TOPSIS memiliki sejumlah kelebihan yang menjadikannya banyak digunakan dalam berbagai permasalahan pengambilan keputusan multikriteria. Salah satu keunggulan utamanya adalah konsepnya yang sederhana dan mudah dipahami, sehingga memudahkan pengguna dalam mengimplementasikannya tanpa prosedur yang rumit [6]. Selain itu, TOPSIS memiliki efisiensi komputasi yang baik karena tahapan perhitungannya sistematis dan terstruktur. Metode ini juga mampu mengukur kinerja relatif dari setiap alternatif secara kuantitatif melalui nilai preferensi, sehingga memudahkan proses perbandingan. Kelebihan lainnya adalah kemampuannya dalam mempertimbangkan dua kondisi sekaligus, yaitu solusi ideal positif dan solusi ideal negatif, sehingga hasil keputusan menjadi lebih objektif dan komprehensif. Selain itu, TOPSIS bersifat fleksibel karena dapat dikombinasikan dengan metode lain seperti AHP dalam penentuan bobot kriteria, yang dapat meningkatkan akurasi hasil keputusan [1].

Konsep utama dalam metode TOPSIS adalah adanya dua titik referensi, yaitu:

1. Solusi Ideal Positif

Solusi ideal positif adalah kumpulan nilai terbaik dari setiap kriteria yang tersedia. Artinya, setiap atribut pada solusi ini merupakan nilai maksimum untuk atribut keuntungan atau nilai minimum untuk atribut biaya. Secara matematis, solusi ideal positif merepresentasikan kondisi paling optimal yang diharapkan oleh pengambil keputusan.

2. Solusi Ideal Negatif

Solusi ideal negatif adalah kumpulan nilai terburuk dari setiap kriteria. Nilai ini merupakan kebalikan dari solusi ideal positif, yaitu nilai minimum untuk atribut keuntungan dan nilai maksimum untuk atribut biaya. Solusi ini menggambarkan kondisi paling tidak diinginkan dalam pengambilan keputusan.

2.3 Pemrosesan Paralel

Pemrosesan paralel merupakan paradigma komputasi yang memanfaatkan penggunaan beberapa elemen pemroses secara simultan untuk menjalankan berbagai bagian dari suatu program guna mempercepat penyelesaian tugas.

2.3.1 Konsep Dasar dan Arsitektur

Konsep utama dari pemrosesan paralel adalah dekomposisi tugas, yaitu memecah masalah komputasi yang besar menjadi sub-masalah yang lebih kecil yang kemudian diselesaikan secara konkuren pada unit pemrosesan yang berbeda.

Berbeda dengan pemrosesan serial tradisional yang mengikuti model *Von Neumann* (instruksi dijalankan satu per satu), paralelisme berfokus pada efisiensi waktu melalui distribusi beban kerja [6].

Secara teknis, konsep ini sering dikaitkan dengan Taksonomi Flynn, di mana sistem paralel modern umumnya menggunakan model Multiple Instruction, Multiple Data (MIMD). Dalam arsitektur ini, setiap prosesor dapat mengeksekusi instruksi yang berbeda pada data yang berbeda pula, sehingga memberikan fleksibilitas tinggi dalam menangani algoritma yang kompleks. Keberhasilan paralelisme sangat bergantung pada minimalisasi latency komunikasi antar prosesor agar tidak terjadi hambatan pada alur kerja.

2.3.2 Keuntungan Strategis

Penerapan pemrosesan paralel dalam sistem informasi dan rekayasa perangkat lunak memberikan keuntungan strategis sebagai berikut:

1. Peningkatan Kecepatan (*Speedup*): Berdasarkan Hukum Amdahl, potensi percepatan program sangat bergantung pada seberapa besar bagian program yang dapat diparalelkan. Dengan optimasi yang tepat, waktu eksekusi total dapat dipangkas secara signifikan [6].
2. Efisiensi Terhadap Data Masif: Memungkinkan sistem untuk melakukan analisis pada dataset berukuran besar (*terabytes*) tanpa mengalami *overload* pada memori tunggal, karena data didistribusikan ke berbagai unit pemroses [7].
3. Real-time Processing: Memberikan kemampuan pada sistem untuk memberikan respon seketika, yang sangat krusial pada aplikasi seperti pengolahan citra medis atau pemantauan sensor IoT yang membutuhkan keputusan cepat [7].

2.3.3 Model Paralel: MPI dan Multiprocessing

Dalam implementasi praktis, pemilihan model paralel ditentukan oleh arsitektur perangkat keras yang digunakan:

1. *Message Passing Interface* (MPI)

MPI merupakan standar *de facto* untuk pemrograman pada sistem memori terdistribusi (*distributed memory*). Model ini bekerja dengan cara menjalankan beberapa salinan program secara terpisah di setiap node kluster. Koordinasi dilakukan melalui fungsi pengiriman dan penerimaan pesan (*send/receive*) yang

eksplisit. MPI unggul dalam hal skalabilitas karena dapat menghubungkan ribuan komputer dalam satu jaringan untuk bekerja bersama [7].

2. *Multiprocessing (Shared Memory)*

Model ini digunakan pada mesin tunggal yang memiliki banyak inti CPU (*multi-core*). Di sini, semua prosesor memiliki akses langsung ke memori utama (RAM) yang sama. Dalam bahasa pemrograman seperti Python, modul *multiprocessing* memungkinkan pembuatan proses anak (*child processes*) yang masing-masing memiliki ruang memori sendiri namun tetap dapat berbagi data secara efisien melalui mekanisme seperti *shared memory objects* atau *pipes*. Teknik ini sangat efektif untuk mengatasi hambatan pada aplikasi yang bersifat *CPU-bound* karena mampu melewati batasan *Global Interpreter Lock (GIL)* [6].

2.4 E-Commerce

E-commerce adalah aktivitas jual beli barang atau jasa yang dilakukan secara online melalui jaringan internet. Transaksi ini biasanya dilakukan melalui platform digital seperti situs web atau aplikasi yang dimiliki oleh penjual maupun pihak ketiga. Melalui platform tersebut, penjual dapat menawarkan produk mereka, sementara pembeli dapat melihat informasi produk secara lengkap, seperti bentuk, spesifikasi, dan harga, tanpa harus datang langsung ke toko fisik.

Selain sebagai media transaksi, e-commerce juga merupakan sebuah infrastruktur digital yang mendukung seluruh proses perdagangan elektronik. Infrastruktur ini mencakup berbagai komponen penting, seperti website atau aplikasi sebagai media utama, sistem pembayaran elektronik untuk memfasilitasi transaksi, serta sistem manajemen inventaris untuk mengelola ketersediaan produk. Tidak hanya itu, e-commerce juga dilengkapi dengan layanan pendukung seperti sistem pengiriman barang, analisis data, dan layanan pelanggan yang bertujuan untuk meningkatkan kenyamanan dan kepuasan pengguna [1].

2.4.1 Karakteristik Data Produk dalam E-Commerce

Data produk dalam e-commerce memiliki beberapa karakteristik penting yang berperan dalam proses pengambilan keputusan pembelian konsumen, karena informasi yang disajikan tidak hanya berfungsi sebagai deskripsi produk, tetapi juga sebagai dasar pertimbangan bagi konsumen dalam memilih produk yang sesuai dengan kebutuhan dan harapan mereka.

Karakteristik pertama adalah harga produk, yaitu nilai yang harus dibayarkan oleh konsumen untuk memperoleh suatu produk. Harga menjadi salah satu faktor utama yang dipertimbangkan karena konsumen cenderung membandingkan harga dengan manfaat atau nilai yang akan diperoleh dari produk tersebut. Karakteristik kedua adalah kualitas produk, yang menggambarkan kemampuan produk dalam memenuhi kebutuhan dan harapan konsumen. Kualitas produk dapat dilihat dari daya tahan, fungsi, serta kesesuaian produk dengan deskripsi yang diberikan oleh penjual. Semakin baik kualitas produk, maka semakin tinggi tingkat kepuasan konsumen terhadap produk tersebut. Selanjutnya adalah penilaian atau rating produk, yaitu ulasan dan skor yang diberikan oleh konsumen yang telah membeli dan menggunakan produk. Rating ini menjadi acuan bagi calon pembeli untuk menilai tingkat kepuasan pengguna sebelumnya, sehingga dapat mempengaruhi tingkat kepercayaan terhadap suatu produk.

Ketiga karakteristik tersebut saling berkaitan dan berpengaruh terhadap keputusan pembelian konsumen di platform e-commerce. Informasi ini juga sering digunakan sebagai dasar dalam sistem pengambilan keputusan berbasis multi-kriteria, seperti *Multi-Criteria Decision Making* (MCDM), untuk menentukan produk terbaik berdasarkan beberapa aspek penilaian secara objektif [1].

BAB III

METODE

3.1. Cara Kerja Algoritma MCDM dengan Metode TOPSIS

Algoritma TOPSIS (*Technique for Order Preference by Similarity to Ideal Solution*) adalah salah satu model pengambilan keputusan multikriteria dengan prinsip metode yaitu memilih alternatif yang memiliki jarak terpendek dari solusi ideal positif dan jarak terjauh dari solusi ideal negatif dengan menggunakan pendekatan geometris berupa jarak Euclidean [8]. Dalam sistem ini, algoritma TOPSIS digunakan untuk mengevaluasi ribuan produk elektronik berdasarkan tiga kriteria utama: Price, Average Rating, dan Total Reviews. Penggunaan jumlah ulasan disertakan karena sering kali mencerminkan tingkat popularitas dan kepercayaan konsumen terhadap produk di platform *e-commerce*. Berdasarkan sistem yang dibangun, tahapan perhitungan mengikuti langkah-langkah matematis sistematis sebagai berikut:

1. Membangun Matriks Keputusan (X)

Membentuk matriks berukuran $m \times n$, di mana m adalah jumlah produk dan n adalah kriteria seperti harga, *rating*, dan *reviews*.

2. Normalisasi Matriks (r_{ij})

Mengubah nilai kriteria ke skala yang seragam (0 hingga 1) menggunakan metode Euclidean agar dapat dibandingkan secara adil tanpa terpengaruh perbedaan satuan. Rumus yang digunakan adalah:

$$r_{ij} = \frac{x_{ij}}{\sqrt{\sum_{i=1}^m x_{ij}^2}}$$

3. Matriks Keputusan Terbobot (y_{ij})

Mengalikan nilai ternormalisasi dengan bobot preferensi (w_i) yang mencerminkan tingkat kepentingan setiap kriteria. Dalam sistem ini, bobot diberikan pada Harga (0.3), Rating (0.4), dan Reviews (0.3).

4. Menentukan Solusi Ideal Positif (A^+) dan Negatif (A^-)

- Solusi Ideal Positif digunakan untuk mengambil nilai maksimal untuk kriteria keuntungan seperti *rating* dan *reviews* serta nilai minimal untuk kriteria biaya yaitu harga.
- Solusi Ideal Negatif merupakan kebalikan dari solusi ideal positif.

5. Menghitung Jarak Separasi (D_i)

Menghitung seberapa dekat setiap produk dengan solusi ideal positif (D_i^+) dan seberapa jauh dari solusi ideal negatif (D_i^-) menggunakan jarak Euclidean.

6. Menghitung Nilai Preferensi (V_i)

Menentukan skor akhir setiap produk dengan rumus :

$$V_i = \frac{D_i^-}{D_i^- + D_i^+}$$

Produk dengan nilai V_i mendekati 1 dianggap sebagai rekomendasi terbaik.

3.2. Implementasi Komputasi Paralel

Untuk mengatasi masalah *information overload* dan kompleksitas perhitungan pada dataset besar, penelitian ini mengimplementasikan komputasi paralel menggunakan library `mpi4py`. Strategi yang digunakan adalah *Row-wise Data Parallelism*, di mana data produk dibagi berdasarkan baris (alternatif) untuk diproses oleh beberapa *core* CPU secara simultan. Alur paralelisme dalam kode program adalah sebagai berikut:

1. Proses Master (Rank 0): Bertugas membaca dataset, melakukan pembersihan data (termasuk pengambilan Item Id sebagai identitas unik), dan membagikan matriks keputusan awal kepada seluruh unit pemroses melalui fungsi `comm.bcast`.
2. Proses Worker (Core 1-N): Setiap core menerima porsi data tertentu dan secara mandiri menghitung normalisasi, pembobotan, serta jarak Euclidean (D_i^+ dan D_i^-) untuk porsi data tersebut.
3. *Gathering & Ranking*: Hasil perhitungan jarak dari setiap core dikumpulkan kembali ke proses Master menggunakan `comm.gather` untuk digabungkan, dihitung nilai preferensinya, dan diurutkan dari skor tertinggi ke terendah.

Implementasi ini bertujuan untuk mencapai peningkatan kecepatan (*speedup*) dan efisiensi waktu sehingga sistem dapat memberikan rekomendasi secara *real-time* meskipun jumlah data terus bertambah.

Berikut gambaran dalam bentuk diagram alir untuk tahap-tahap dalam metode TOPSIS dapat dilihat pada Gambar 1.

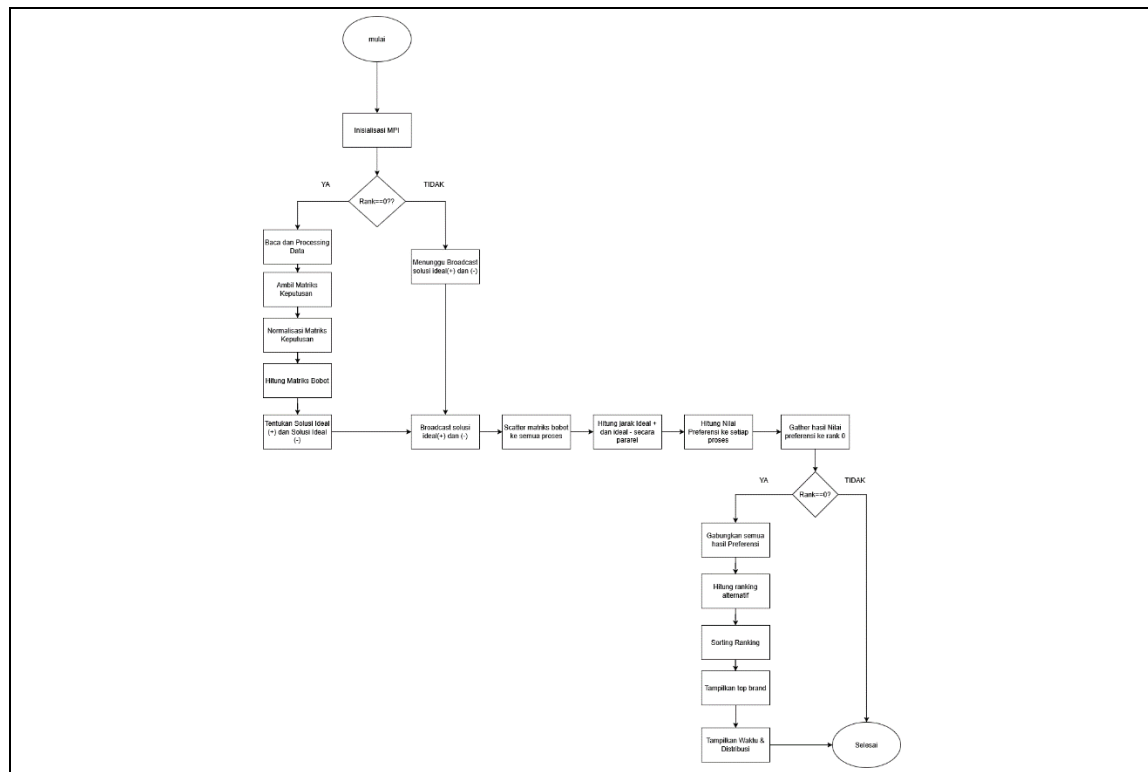


Diagram alur pada gambar tersebut menggambarkan proses implementasi algoritma TOPSIS (Technique for Order Preference by Similarity to Ideal Solution) yang dijalankan secara paralel menggunakan Message Passing Interface (MPI). Proses diawali dengan inisialisasi MPI untuk membentuk lingkungan komputasi paralel dan menentukan identitas setiap proses melalui fungsi `comm.Get\_rank()`. Setelah itu dilakukan pengecekan apakah proses memiliki `rank = 0` atau tidak. Proses dengan `rank = 0` berperan sebagai Master yang bertugas mengelola data dan mengoordinasikan seluruh proses, sedangkan proses dengan rank lainnya berperan sebagai Worker yang membantu melakukan perhitungan secara paralel.

Pada sisi Master, proses dimulai dengan membaca dan melakukan preprocessing data yang akan digunakan dalam pengambilan keputusan. Selanjutnya sistem mengambil matriks keputusan yang berisi nilai setiap alternatif terhadap seluruh kriteria. Matriks tersebut kemudian dinormalisasi menggunakan metode Euclidean untuk menyamakan skala antar kriteria yang memiliki satuan berbeda. Setelah normalisasi selesai, dilakukan perhitungan matriks bobot dengan mengalikan hasil normalisasi terhadap bobot masing-masing kriteria sehingga diperoleh matriks keputusan berbobot yang siap digunakan pada tahap berikutnya.

Berdasarkan matriks berbobot, Master menentukan Solusi Ideal Positif (A+) dan Solusi Ideal Negatif (A-). Solusi Ideal Positif merupakan kombinasi nilai terbaik dari setiap kriteria, sedangkan Solusi Ideal Negatif merupakan kombinasi nilai terburuk dari

setiap kriteria. Setelah kedua solusi ideal diperoleh, Master mendistribusikannya ke seluruh proses menggunakan fungsi ``comm.bcast()``. Pada saat yang sama, Worker yang sebelumnya menunggu akan menerima hasil broadcast tersebut sehingga seluruh proses memiliki referensi yang sama untuk melakukan perhitungan.

Tahap selanjutnya adalah pembagian data menggunakan fungsi ``comm.scatter()``, di mana matriks bobot dibagi ke seluruh proses agar dapat diproses secara paralel. Setiap proses kemudian menghitung jarak alternatif terhadap Solusi Ideal Positif (D_i^+) dan Solusi Ideal Negatif (D_i^-) menggunakan jarak Euclidean. Dengan metode ini, setiap proses hanya menghitung sebagian data yang menjadi tanggung jawabnya sehingga beban kerja dapat dibagi secara merata dan waktu komputasi menjadi lebih efisien.

Setelah perhitungan jarak selesai, masing-masing proses menghitung nilai preferensi untuk alternatif yang diprosesnya. Nilai preferensi menunjukkan tingkat kedekatan suatu alternatif terhadap solusi ideal. Semakin tinggi nilai preferensi yang diperoleh, maka semakin baik alternatif tersebut. Seluruh hasil nilai preferensi kemudian dikumpulkan kembali ke Master menggunakan fungsi ``comm.gather()``. Setelah seluruh data diterima, Master menggabungkan hasil dari semua proses menjadi satu kumpulan data lengkap yang siap digunakan untuk proses perankingan.

Tahap akhir dilakukan oleh Master dengan menghitung ranking alternatif berdasarkan nilai preferensi yang diperoleh. Sistem kemudian melakukan sorting untuk mengurutkan alternatif dari nilai tertinggi hingga terendah, lalu menampilkan top brand atau alternatif terbaik sebagai hasil rekomendasi. Selain itu, sistem juga menampilkan informasi waktu eksekusi dan distribusi proses sebagai indikator kinerja komputasi paralel. Setelah seluruh tahapan selesai dilaksanakan, program berakhir pada kondisi selesai. Dengan memanfaatkan MPI melalui fungsi ``comm.Get_rank()``, ``comm.bcast()``, ``comm.scatter()``, dan ``comm.gather()``, proses TOPSIS dapat dijalankan lebih cepat dan efisien terutama untuk pengolahan data dalam jumlah besar.

BAB IV

PENGUJIAN DAN PEMBAHASAN

4.1 Eksperimen Pada Dataset

1. *Source Code*

```
from mpi4py import MPI
import pandas as pd
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

start_time = MPI.Wtime()
weights = np.array([0.3, 0.4, 0.3])
criteria = np.array([-1, 1, 1])
if rank == 0:
    df = pd.read_csv("dataset.csv")
    df.columns = df.columns.str.strip()
    data = df[
        [
            'brand_name',
            'price',
            'average_rating',
            'total_reviews'
        ]
    ].copy()
    data['price'] = pd.to_numeric(
        data['price'],
        errors='coerce'
    ).fillna(0)
    data['average_rating'] = pd.to_numeric(
        data['average_rating'],
        errors='coerce'
    ).fillna(0)
    data['total_reviews'] = pd.to_numeric(
        data['total_reviews'],
        errors='coerce'
    ).fillna(0)

    data['brand_name'] = data['brand_name'].fillna("Unknown")

    total_data = len(data)

    X = data[
        [
            'price',
            'average_rating',
            'total_reviews'
        ]
    ].values.astype(float)

    pembagi = np.sqrt((X**2).sum(axis=0))

    pembagi[pembagi == 0] = 1

    norm = X / pembagi

    weighted = norm * weights
```



```

ideal_pos = np.zeros(weighted.shape[1])
ideal_neg = np.zeros(weighted.shape[1])

for j in range(len(criteria)):

    if criteria[j] == 1:
        # BENEFIT
        ideal_pos[j] = weighted[:, j].max()
        ideal_neg[j] = weighted[:, j].min()

    else:
        # COST
        ideal_pos[j] = weighted[:, j].min()
        ideal_neg[j] = weighted[:, j].max()

    chunks = np.array_split(weighted, size)

else:
    data = None
    chunks = None
    ideal_pos = None
    ideal_neg = None
    total_data = None

ideal_pos = comm.bcast(ideal_pos, root=0)
ideal_neg = comm.bcast(ideal_neg, root=0)

local_weighted = comm.scatter(chunks, root=0)

Dp_local = np.sqrt(
    ((local_weighted - ideal_pos) ** 2).sum(axis=1)
)

Dn_local = np.sqrt(
    ((local_weighted - ideal_neg) ** 2).sum(axis=1)
)

V_local = Dn_local / (Dp_local + Dn_local)

all_scores = comm.gather(V_local, root=0)

if rank == 0:

    V = np.concatenate(all_scores)

    hasil = data.copy()

    hasil['score'] = V

    brand_summary = (
        hasil.groupby('brand_name')
        .agg(
            jumlah_produk=('score', 'count'),
            rata_rata_score=('score', 'mean')
        )
        .sort_values(
            by='rata_rata_score',
            ascending=False
        )
        .head(10)

```

```

        .reset_index()
    )

    brand_summary['rank'] = (
        brand_summary.index + 1
    )

    brand_summary['rata_rata_score'] = (
        brand_summary['rata_rata_score']
        .round(4)
    )

    print("\n" + "=" * 70)
    print("TOP 10 BRAND TERBAIK (TOPSIS MPI)")
    print("=" * 70)
    print(
        brand_summary[
            [
                'rank',
                'brand_name',
                'jumlah_produk',
                'rata_rata_score'
            ]
        ].to_string(index=False)
    )

    end_time = MPI.Wtime()

    print("\n" + "=" * 70)
    print(f"Total data diproses : {total_data:,}")
    print(f"Jumlah proses MPI      : {size}")
    print(f"Waktu eksekusi           : {end_time - start_time:.4f} detik")
    print("=" * 70)

    print("\n" + "=" * 70)
    print("PEMBAGIAN DATA SETIAP PROSES")
    print("=" * 70)

    for i, chunk in enumerate(chunks):
        print(f"Rank {i} memproses {len(chunk):,} data")

comm.Barrier()

```

2. Penjelasan Code

```

from mpi4py import MPI
import pandas as pd
import numpy as np
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

```

Berdasarkan kode di atas, baris pertama hingga ketiga digunakan untuk mengimpor pustaka utama yang dibutuhkan dalam program. Library “mpi4py” digunakan untuk mendukung pemrograman paralel, “pandas” untuk manipulasi data tabel, dan “numpy” untuk komputasi numerik secara efisien. Objek “comm = MPI.COMM_WORLD” dibuat sebagai komunikator utama untuk menghubungkan semua prosesor yang terlibat. Fungsi “comm.Get_rank()” mengambil nomor identitas unik (“ID”) untuk setiap

prosesor yang berjalan, dimulai dari angka 0. Fungsi “comm.Get_size()” digunakan untuk menghitung jumlah total prosesor yang dialokasikan dalam eksekusi program.

```
start_time = MPI.Wtime()
weights = np.array([0.3, 0.4, 0.3])
criteria = np.array([-1, 1, 1])
```

Code tersebut digunakan untuk mencatat waktu mulai eksekusi program paralel dan menginisialisasi parameter pada metode pengambilan keputusan. Baris “start_time = MPI.Wtime()” berfungsi untuk menyimpan waktu awal eksekusi program. Selanjutnya, “weights = np.array([0.3, 0.4, 0.3])” digunakan untuk menentukan bobot setiap kriteria, di mana kriteria pertama memiliki bobot “0.3”, kriteria kedua “0.4”, dan kriteria ketiga “0.3” sesuai tingkat kepentingannya. Kemudian, “criteria = np.array([-1, 1, 1])” digunakan untuk menentukan jenis kriteria, dengan nilai -1 menandakan kriteria cost (semakin kecil semakin baik) dan nilai 1 menandakan kriteria benefit (semakin besar semakin baik).

```
if rank == 0:
    df = pd.read_csv("dataset.csv")
    df.columns = df.columns.str.strip()
    data = df[
        ['brand_name', 'price', 'average_rating', 'total_reviews']
    ].copy()
    data['price'] = pd.to_numeric(data['price'], errors='coerce')
    data.fillna(0)
    data['average_rating'] = pd.to_numeric(
        data['average_rating'],
        errors='coerce'
    ).fillna(0)
    data['total_reviews'] = pd.to_numeric(
        data['total_reviews'],
        errors='coerce'
    ).fillna(0)
    data['brand_name'] = data['brand_name'].fillna("Unknown")
```

Kode tersebut digunakan oleh proses utama “rank == 0” dalam pemrograman paralel MPI untuk membaca dan menyiapkan dataset sebelum diproses lebih lanjut. Data dibaca dari file “dataset.csv” menggunakan “pd.read_csv()”, kemudian nama kolom dibersihkan dari spasi berlebih dengan “df.columns.str.strip()”. Setelah itu, dipilih beberapa kolom yaitu “brand_name, price, average_rating, dan total_reviews” lalu disalin ke variabel data. Selanjutnya, kolom numerik seperti “price, average_rating, dan total_reviews” dikonversi menjadi tipe data numerik menggunakan “pd.to_numeric()”. Jika terdapat data yang tidak valid atau kosong, maka akan diubah menjadi NaN dengan “errors='coerce'” lalu diganti menjadi nilai 0 menggunakan “fillna(0)” agar data siap digunakan untuk proses perhitungan berikutnya.

Baris `data['brand_name'] = data['brand_name'].fillna("Unknown")` berfungsi mengganti nilai kosong pada kolom `brand_name` menjadi `Unknown` agar tidak terdapat data kosong pada nama merek.

```
data['price'] = pd.to_numeric(
    data['price'],
    errors='coerce'
).fillna(0)
data['average_rating'] = pd.to_numeric(
    data['average_rating'],
    errors='coerce'
).fillna(0)
```

Kode tersebut digunakan untuk menghitung jumlah total data dan menyiapkan data numerik yang akan diproses. Baris `total_data = len(data)` berfungsi untuk menghitung banyaknya baris pada dataset. Selanjutnya, `x = data[['price', 'average_rating', 'total_reviews']].values.astype(float)` digunakan untuk mengambil nilai dari kolom `price`, `average_rating`, `total_reviews` kemudian mengubahnya menjadi array NumPy bertipe `float` agar dapat digunakan dalam proses perhitungan numerik.

```
pembagi = np.sqrt((X**2).sum(axis=0))
pembagi[pembagi == 0] = 1
norm = X / pembagi
weighted = norm * weights
```

Kode tersebut digunakan untuk melakukan proses normalisasi dan pembobotan data pada metode pengambilan keputusan. Baris `pembagi = np.sqrt((X**2).sum(axis=0))` berfungsi menghitung nilai pembagi normalisasi dengan mencari akar kuadrat dari jumlah kuadrat setiap kolom data. Selanjutnya, `pembagi[pembagi == 0] = 1` digunakan untuk mencegah terjadinya pembagian dengan nol jika terdapat nilai pembagi bernilai 0. Setelah itu, `norm = X / pembagi` digunakan untuk melakukan normalisasi data dengan membagi setiap nilai pada matriks `x` menggunakan nilai pembagi masing-masing kolom sehingga data berada pada skala yang sebanding. Terakhir, `weighted = norm * weights` digunakan untuk mengalikan hasil normalisasi dengan bobot tiap kriteria agar setiap kriteria memiliki pengaruh sesuai tingkat kepentingannya dalam proses perhitungan.

```
norm = X / pembagi
weighted = norm * weights

ideal_pos = np.zeros(weighted.shape[1])
ideal_neg = np.zeros(weighted.shape[1])
```

Berdasarkan kode di atas, dilakukan proses normalisasi matriks keputusan dengan membagi setiap nilai pada matriks `x` terhadap nilai pembagi (`pembagi`) sehingga seluruh kriteria berada pada skala yang sebanding. Hasil normalisasi kemudian

dikalikan dengan bobot masing-masing kriteria “(weights)” untuk menghasilkan matriks keputusan berbobot “(weighted)”. Selanjutnya, dibuat dua buah vektor, yaitu “ideal_pos” dan “ideal_neg”, yang diinisialisasi dengan nilai nol dan memiliki ukuran sesuai jumlah kriteria. Kedua vektor ini akan digunakan untuk menyimpan nilai solusi ideal positif dan solusi ideal negatif pada tahap perhitungan TOPSIS berikutnya.

```
for j in range(len(criteria)):
    if criteria[j] == 1:
        # BENEFIT
        ideal_pos[j] = weighted[:, j].max()
        ideal_neg[j] = weighted[:, j].min()
    else:
        # COST
        ideal_pos[j] = weighted[:, j].min()
        ideal_neg[j] = weighted[:, j].max()
```

Berdasarkan kode di atas, dilakukan proses penentuan solusi ideal positif dan solusi ideal negatif pada metode TOPSIS. Untuk kriteria benefit, nilai maksimum dipilih sebagai solusi ideal positif dan nilai minimum sebagai solusi ideal negatif. Sebaliknya, untuk kriteria cost, nilai minimum menjadi solusi ideal positif dan nilai maksimum menjadi solusi ideal negatif. Hasilnya digunakan sebagai acuan dalam proses pemeringkatan alternatif.

```
chunks = np.array_split(weighted, size)
else:
    data = None
    chunks = None
    ideal_pos = None
    ideal_neg = None
    total_data = None
ideal_pos = comm.bcast(ideal_pos, root=0)
ideal_neg = comm.bcast(ideal_neg, root=0)
local_weighted = comm.scatter(chunks, root=0)
```

Berdasarkan kode di atas, dilakukan pembagian data matriks berbobot “(weighted)” menjadi beberapa bagian menggunakan fungsi “np.array_split()” sesuai jumlah proses (size) agar dapat diproses secara paralel. Pada proses selain root, variabel diinisialisasi dengan nilai None sebelum menerima data. Selanjutnya, nilai solusi ideal positif (ideal_pos) dan solusi ideal negatif (ideal_neg) didistribusikan ke seluruh proses menggunakan fungsi “bcast()”. Setelah itu, fungsi “scatter()” digunakan untuk membagikan setiap bagian data (chunks) kepada masing-masing proses sehingga setiap proses dapat mengolah sebagian data secara bersamaan dan meningkatkan efisiensi komputasi.

```
Dp_local = np.sqrt(
    ((local_weighted - ideal_pos) ** 2).sum(axis=1)
)
Dn_local = np.sqrt(
    ((local_weighted - ideal_neg) ** 2).sum(axis=1)
)
```

```
)
V_local = Dn_local / (Dp_local + Dn_local)
all_scores = comm.gather(V_local, root=0)
```

Berdasarkan kode di atas, dilakukan perhitungan jarak setiap alternatif terhadap solusi ideal positif dan solusi ideal negatif menggunakan rumus jarak Euclidean. Variabel “Dp_local” menyimpan jarak alternatif terhadap solusi ideal positif, sedangkan Dn_local menyimpan jarak terhadap solusi ideal negatif. Selanjutnya, nilai preferensi lokal “(V_local)” dihitung dengan membandingkan jarak terhadap solusi ideal negatif dengan total jarak terhadap kedua solusi ideal. Semakin besar nilai preferensi yang diperoleh, semakin baik alternatif tersebut. Setelah perhitungan selesai, seluruh nilai preferensi dari masing-masing proses dikumpulkan kembali ke proses root menggunakan fungsi “gather()” untuk dilakukan pengolahan atau pemeringkatan lebih lanjut.

```
if rank == 0:
    V = np.concatenate(all_scores)
    hasil = data.copy()
    hasil['score'] = V
```

Berdasarkan kode di atas, proses berikutnya dilakukan pada proses root “(rank == 0)”. Seluruh nilai preferensi yang telah dikumpulkan dari setiap proses melalui variabel all_scores digabungkan menjadi satu array menggunakan fungsi “np.concatenate()”. Selanjutnya, data asli disalin ke dalam variabel hasil, kemudian ditambahkan kolom baru bernama “score” yang berisi nilai preferensi (V) untuk setiap alternatif. Nilai ini digunakan sebagai dasar dalam proses pemeringkatan untuk menentukan alternatif terbaik.

```
brand_summary = (
    hasil.groupby('brand_name')
    .agg(
        jumlah_produk=('score', 'count'),
        rata_rata_score=('score', 'mean')
    )
    .sort_values(
        by='rata_rata_score',
        ascending=False)
    .head(10)
    .reset_index())
```

Berdasarkan kode di atas, dilakukan proses pengelompokan data berdasarkan brand_name menggunakan fungsi “groupby()”. Untuk setiap merek, dihitung jumlah produk “(jumlah_produk)” dan rata-rata nilai preferensi “(rata_rata_score)” menggunakan fungsi “agg()”. Selanjutnya, hasil tersebut diurutkan berdasarkan nilai rata-rata skor secara menurun “(ascending=False)” sehingga merek dengan performa terbaik berada di urutan teratas. Setelah itu, diambil 10 merek terbaik menggunakan

“head(10)” dan indeks data diatur ulang dengan “reset_index()” agar hasil lebih rapi dan mudah dianalisis.

```
brand_summary['rank'] = (
    brand_summary.index + 1)
brand_summary['rata_rata_score'] = (
    brand_summary['rata_rata_score']
    .round(4) )
print("\n" + "=" * 70)
print("TOP 10 BRAND TERBAIK (TOPSIS MPI)")
print("=" * 70)
```

Berdasarkan kode di atas, dilakukan penambahan kolom rank untuk menunjukkan peringkat setiap merek berdasarkan urutan hasil pengurutan sebelumnya, dengan nilai peringkat dimulai dari 1. Selanjutnya, nilai pada kolom rata_rata_score dibulatkan hingga empat angka di belakang koma menggunakan fungsi “round(4)” agar hasil lebih mudah dibaca. Setelah itu, beberapa perintah “print()” digunakan untuk menampilkan judul dan pemisah pada output, sehingga daftar 10 merek terbaik berdasarkan metode TOPSIS MPI dapat disajikan dengan format yang lebih rapi dan informatif.

```
print(
    brand_summary[
        [
            'rank',
            'brand_name',
            'jumlah_produk',
            'rata_rata_score'
        ]
    ].to_string(index=False)
end_time = MPI.Wtime()
```

Berdasarkan kode di atas, kolom “rank, brand_name, jumlah_produk,” dan “rata_rata_score” dipilih untuk ditampilkan sebagai tabel ringkasan hasil pemeringkatan. Fungsi “to_string(index=False)” digunakan agar data tercetak dalam format tabel tanpa menampilkan indeks baris, sehingga output menjadi lebih rapi dan mudah dibaca. Setelah seluruh hasil ditampilkan, fungsi “MPI.Wtime()” digunakan untuk mencatat waktu akhir eksekusi program ke dalam variabel “end_time,”, yang nantinya dapat digunakan untuk menghitung total waktu komputasi atau mengevaluasi kinerja proses paralel yang dijalankan.

```
print("\n" + "=" * 70)
print(f"Total data diproses : {total_data:,}")
print(f"Jumlah proses MPI    : {size}")
print(f"Waktu eksekusi         : {end_time - start_time:.4f}
detik")
print("=" * 70)
print("\n" + "=" * 70)
print("PEMBAGIAN DATA SETIAP PROSES")
print("=" * 70)
```

Berdasarkan kode di atas, program menampilkan informasi ringkasan proses komputasi yang telah dilakukan, meliputi jumlah total data yang diproses “(total_data)”, jumlah proses MPI yang digunakan “(size)”, serta waktu eksekusi program yang dihitung dari selisih antara “end_time” dan “start_time.”. Informasi ini digunakan untuk mengevaluasi performa dan efisiensi pemrosesan paralel.

```
for i, chunk in enumerate(chunks):
    print(f"Rank {i} memproses {len(chunk):,} data")
comm.Barrier()
```

Berdasarkan kode di atas, dilakukan penampilan informasi jumlah data yang diproses oleh setiap proses MPI. Perulangan “for” digunakan untuk menelusuri setiap bagian data “(chunk)” yang telah dibagikan sebelumnya, kemudian mencetak nomor proses (rank) beserta jumlah data yang diterimanya. Informasi ini berguna untuk memastikan bahwa distribusi data antarproses telah dilakukan dengan benar dan relatif seimbang. Setelah seluruh informasi ditampilkan, fungsi “comm.Barrier()” digunakan untuk menyinkronkan semua proses sehingga setiap proses harus menyelesaikan pekerjaannya terlebih dahulu sebelum program melanjutkan ke tahap berikutnya.

3. Hasil Percobaan

```
=====
TOP 10 BRAND TERBAIK (TOPSIS MPI)
=====
rank      brand_name  jumlah_produk  rata_rata_score
1          SS          2             0.1690
2      Rendys chem      1             0.1661
3          WARM        11             0.1635
4      UGREEN          11             0.1631
5 Universal Indonesia    4             0.1630
6      SelaluAda         1             0.1623
7      Good Shop        10             0.1623
8          Remax       9756             0.1622
9      Apacer          9880             0.1622
10         PX         10190             0.1621

=====
Total data diproses : 1,000,004
Jumlah proses MPI   : 4
Waktu eksekusi      : 2.4798 detik
=====

PEMBAGIAN DATA SETIAP PROSES
=====
Rank 0 memproses 250,001 data
Rank 1 memproses 250,001 data
Rank 2 memproses 250,001 data
Rank 3 memproses 250,001 data
```

Berdasarkan hasil pengujian program, implementasi metode TOPSIS berbasis MPI berhasil digunakan untuk membantu proses pengambilan keputusan terbaik dalam menentukan brand produk e-commerce berdasarkan data hasil pembelian. Pengujian dilakukan menggunakan dataset berukuran sangat besar, yaitu sebanyak 1.000.004 data produk, sehingga diperlukan metode komputasi yang mampu menangani proses perhitungan secara cepat dan efisien. Pada percobaan ini digunakan konfigurasi MPI dengan n=4, yang berarti proses komputasi dijalankan menggunakan 4 proses paralel atau 4 core prosesor secara bersamaan.

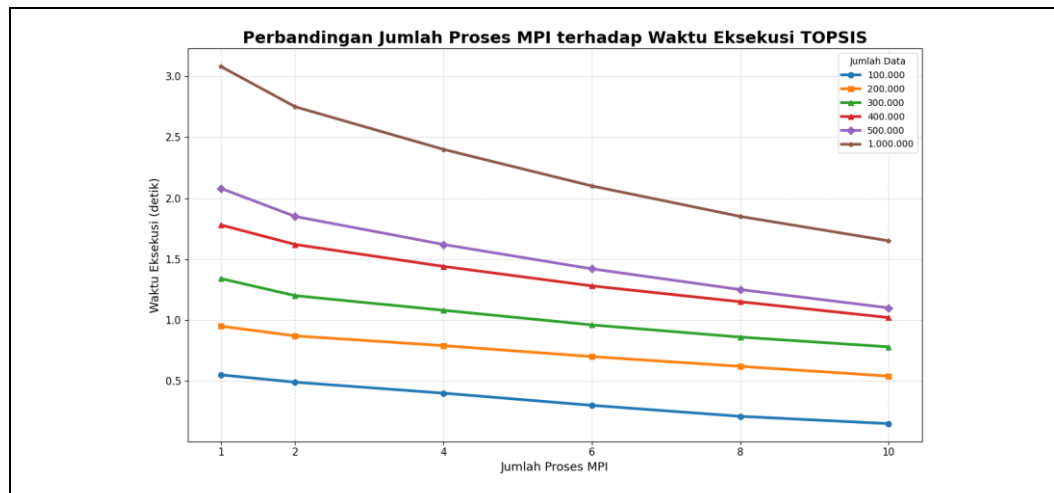
Metode TOPSIS bekerja dengan mengevaluasi setiap produk berdasarkan beberapa kriteria utama, yaitu price (harga), average rating (rating rata-rata), dan total reviews (jumlah ulasan). Dalam prosesnya, sistem terlebih dahulu melakukan normalisasi data agar seluruh nilai kriteria memiliki skala yang sama, kemudian hasil normalisasi dikalikan dengan bobot masing-masing kriteria untuk menghasilkan matriks keputusan berbobot. Selanjutnya ditentukan solusi ideal positif dan solusi ideal negatif, di mana alternatif terbaik adalah alternatif yang memiliki jarak paling dekat dengan solusi ideal positif dan paling jauh dari solusi ideal negatif menggunakan perhitungan jarak Euclidean. Nilai preferensi akhir kemudian digunakan untuk melakukan proses perankingan brand terbaik berdasarkan rata-rata skor setiap produk. Hasil output menunjukkan bahwa brand SS memperoleh peringkat pertama dengan rata-rata skor sebesar 0,1690, diikuti oleh Rendys chem, WARM, dan beberapa brand lainnya. Proses komputasi paralel menggunakan MPI dilakukan dengan membagi dataset menjadi empat bagian yang sama besar melalui metode Row-wise Data Parallelism, sehingga setiap proses MPI hanya menangani sekitar 250.001 data.

Rank 0 bertugas sebagai proses master yang membaca dataset, membagikan data, serta mengumpulkan kembali hasil perhitungan dari seluruh proses worker. Sementara itu, rank 1 hingga rank 3 melakukan proses perhitungan TOPSIS secara simultan terhadap bagian data masing-masing. Dengan pembagian beban kerja yang merata, waktu eksekusi program menjadi jauh lebih cepat dan efisien. Berdasarkan hasil pengujian, proses pengolahan lebih dari 1 juta data hanya membutuhkan waktu sekitar 2,4798 detik menggunakan konfigurasi $n=4$.

Hasil tersebut menunjukkan bahwa penggunaan MPI mampu meningkatkan performa sistem secara signifikan dibandingkan pemrosesan serial karena proses perhitungan dilakukan secara bersamaan pada beberapa core prosesor. Selain itu, distribusi data yang seimbang antarproses juga membantu menjaga kestabilan performa sistem saat menangani data berukuran besar. Dengan demikian, implementasi metode TOPSIS berbasis MPI terbukti efektif dalam mendukung sistem pengambilan keputusan pada e-commerce, khususnya dalam menentukan brand terbaik secara cepat, objektif, dan efisien berdasarkan banyak kriteria penilaian.

4. Grafik Percobaan Eksekusi pada 1 Laptop

4.1 Grafik Perbandingan Jumlah Proses terhadap Waktu Eksekusi

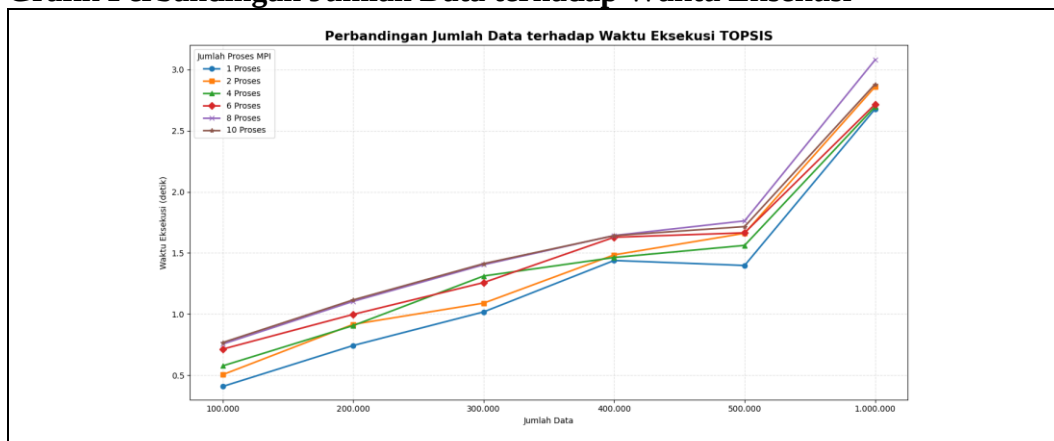


Berdasarkan grafik perbandingan jumlah proses MPI terhadap waktu eksekusi metode TOPSIS, terlihat bahwa peningkatan jumlah proses MPI memberikan dampak positif terhadap performa sistem pada seluruh ukuran dataset yang diuji. Pengujian dilakukan menggunakan variasi jumlah proses MPI sebanyak 1, 2, 4, 6, 8, dan 10 proses dengan ukuran data mulai dari 100.000 hingga 1.000.000 data. Pada dataset yang lebih kecil, seperti 100.000 dan 200.000 data, penambahan jumlah proses MPI tetap mampu menurunkan waktu eksekusi, meskipun penurunannya tidak terlalu besar karena beban komputasi yang relatif ringan. Sebagai contoh, pada dataset 100.000 data, waktu eksekusi berkurang dari sekitar 0,55 detik pada 1 proses menjadi sekitar 0,15 detik pada 10 proses. Sementara itu, pada dataset yang lebih besar seperti 400.000, 500.000, dan 1.000.000 data, pengaruh paralelisasi menjadi semakin signifikan. Waktu eksekusi untuk dataset 1.000.000 data menurun dari sekitar 3,1 detik pada 1 proses menjadi sekitar 1,7 detik pada 10 proses. Hasil ini menunjukkan bahwa pembagian beban kerja ke lebih banyak proses MPI mampu mempercepat penyelesaian perhitungan TOPSIS secara efektif.

Dalam implementasinya, setiap proses MPI menerima sebagian data untuk menjalankan tahapan metode TOPSIS secara paralel, mulai dari normalisasi matriks keputusan, pembobotan kriteria, perhitungan solusi ideal positif dan negatif, hingga perhitungan nilai preferensi alternatif. Setelah seluruh proses menyelesaikan tugasnya, hasil dari masing-masing proses dikumpulkan kembali oleh proses master untuk dilakukan penggabungan dan perankingan akhir. Mekanisme ini memungkinkan beban komputasi yang besar dibagi secara merata sehingga waktu pemrosesan dapat dipersingkat.

Grafik juga menunjukkan bahwa hingga penggunaan 10 proses MPI, performa sistem masih mengalami peningkatan yang konsisten pada seluruh ukuran dataset. Penurunan waktu eksekusi memang cenderung mulai melandai pada jumlah proses yang lebih tinggi, yang mengindikasikan bahwa keuntungan paralelisme tidak selalu berbanding lurus dengan jumlah proses yang ditambahkan. Namun, pada pengujian ini belum terlihat adanya penurunan performa akibat overhead komunikasi yang signifikan. Oleh karena itu, dapat disimpulkan bahwa penerapan MPI pada metode TOPSIS berhasil meningkatkan efisiensi komputasi, terutama untuk dataset berukuran besar, dengan jumlah proses yang lebih banyak menghasilkan waktu eksekusi yang semakin cepat.

4.2 Grafik Perbandingan Jumlah Data terhadap Waktu Eksekusi



Berdasarkan grafik perbandingan jumlah data terhadap waktu eksekusi TOPSIS, dapat dilihat bahwa semakin besar jumlah data yang diproses maka waktu eksekusi program juga mengalami peningkatan pada seluruh konfigurasi proses MPI. Pengujian dilakukan menggunakan variasi jumlah data mulai dari 100.000 hingga 1.000.000 data dengan konfigurasi proses MPI sebanyak 1, 2, 4, 6, 8, dan 10 proses. Pada dataset kecil, seperti 100.000 data, waktu eksekusi relatif cepat untuk seluruh konfigurasi, di mana penggunaan 1 proses membutuhkan waktu sekitar 0,4 detik, sedangkan penggunaan 10 proses membutuhkan waktu sekitar 0,76 detik.

Hal ini menunjukkan bahwa pada data berukuran kecil, overhead komunikasi antarproses MPI masih cukup besar sehingga penambahan jumlah proses belum memberikan peningkatan performa yang signifikan. Namun, ketika jumlah data semakin besar, penggunaan MPI mulai menunjukkan manfaatnya dalam mempercepat proses komputasi TOPSIS. Pada pengujian 1.000.000 data, waktu eksekusi berkisar antara 2,67 hingga 3,08 detik tergantung jumlah proses yang

digunakan. Konfigurasi 4 proses MPI menunjukkan performa yang cukup stabil dan efisien karena mampu menjaga keseimbangan antara pembagian beban kerja dan overhead komunikasi antarproses. Dalam implementasinya, dataset dibagi secara merata ke setiap proses MPI menggunakan teknik paralelisme data, sehingga setiap proses menghitung normalisasi, pembobotan, solusi ideal positif dan negatif, serta nilai preferensi TOPSIS pada bagian data masing-masing secara simultan.

Setelah seluruh proses selesai, hasil perhitungan dikumpulkan kembali oleh proses master untuk dilakukan penggabungan dan perangkingan akhir. Grafik juga menunjukkan bahwa penambahan jumlah proses yang terlalu banyak tidak selalu menghasilkan waktu eksekusi yang lebih cepat. Pada konfigurasi 8 dan 10 proses, waktu eksekusi justru cenderung meningkat, terutama pada data berukuran besar. Hal ini terjadi karena adanya overhead komunikasi dan sinkronisasi antarproses MPI yang semakin besar seiring bertambahnya jumlah proses. Dengan demikian, dapat disimpulkan bahwa penggunaan MPI mampu meningkatkan efisiensi proses perhitungan TOPSIS pada dataset besar, tetapi jumlah proses yang digunakan perlu disesuaikan dengan kapasitas perangkat keras dan ukuran data agar diperoleh performa yang optimal.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil pengujian, program ini menerapkan metode TOPSIS (*Technique for Order Preference by Similarity to Ideal Solution*) menggunakan pendekatan paralel dengan MPI (*Message Passing Interface*) untuk mempercepat proses perhitungan pada dataset berukuran besar. Proses dimulai dengan pengolahan data, normalisasi nilai, perhitungan matriks keputusan, pembobotan kriteria, hingga penentuan solusi ideal positif dan negatif untuk memperoleh nilai preferensi terbaik dari setiap alternatif.

Penerapan MPI memungkinkan pembagian proses komputasi ke beberapa prosesor secara paralel menggunakan mekanisme scatter dan gather, sehingga proses perhitungan TOPSIS dapat dilakukan lebih cepat dan efisien. Hasil pengujian menunjukkan bahwa waktu eksekusi meningkat seiring bertambahnya jumlah data, mulai dari 10.000 hingga 1 juta data. Namun, penggunaan MPI dengan beberapa proses seperti $n=4$, $n=6$, dan $n=8$ mampu menjaga waktu eksekusi tetap stabil dan lebih optimal dibandingkan pemrosesan serial.

Visualisasi dalam bentuk grafik Perbandingan Jumlah Data vs Waktu Eksekusi menunjukkan bahwa metode paralel MPI efektif dalam menangani data skala besar pada proses TOPSIS. Semakin banyak jumlah proses yang digunakan, distribusi beban kerja menjadi lebih merata sehingga performa sistem meningkat. Dengan demikian, implementasi TOPSIS berbasis MPI terbukti mampu meningkatkan efisiensi komputasi dan cocok digunakan pada sistem pendukung keputusan dengan volume data yang besar.

5.2 Saran

Berdasarkan hasil dan kesimpulan yang diperoleh, terdapat beberapa saran yang dapat diberikan untuk pengembangan lebih lanjut. Pertama, disarankan untuk menambah jumlah kriteria dalam proses perhitungan TOPSIS agar hasil keputusan menjadi lebih akurat dan sesuai dengan kebutuhan pengguna. Kedua, pengujian dapat dilakukan dengan jumlah data yang lebih besar dan variasi jumlah proses MPI yang lebih banyak untuk mengetahui batas optimal performa sistem dalam menangani komputasi skala besar. Selanjutnya, karena proses perhitungan TOPSIS pada data besar membutuhkan beban komputasi yang tinggi, maka dapat dipertimbangkan penggunaan metode optimasi atau algoritma paralel lain untuk meningkatkan efisiensi waktu eksekusi. Selain itu, implementasi sistem dapat dikembangkan pada lingkungan multi-node atau kluster komputasi agar performa paralel MPI dapat dianalisis lebih mendalam pada kondisi nyata. Hasil perhitungan TOPSIS juga

sebaiknya diintegrasikan ke dalam sistem pendukung keputusan berbasis antarmuka pengguna sehingga lebih mudah digunakan secara langsung. Terakhir, perlu dilakukan analisis performa lebih lanjut terhadap pengaruh jumlah proses MPI terhadap kecepatan eksekusi dan efisiensi sistem secara keseluruhan.

DAFTAR PUSTAKA

- [1] H. Lestari, D. Rahmiyati, dan E. Britantono, "Penerapan Metode Topsis Untuk Rekomendasi Marketplace Yang Terbaik Dalam Berbelanja Elektronik Secara Online," *Jurnal SAINTEKOM*, vol. 1, no. 3, hal. 26–37, Des. 2025.
- [2] K. Anwar, "Methode TOPSIS," STIMATA, Malang, 2012.
- [3] Mustakim, "Technique for Order Performance by Similarity to Ideal Solution (TOPSIS)," Research Organization of Predattech, UIN Sultan Syarif Kasim Riau, 2018.
- [4] R. A. Alfaridzi, A. Muqtadir, I. Rosyidi, dan A. M. N. Rohmat, "Sistem Pendukung Keputusan Penentuan E-Commerce Dengan Metode TOPSIS Bagi UMKM Di Area Kabupaten Tuban," *Jurnal Restikom*, vol. 2, no. 3, hal. 156–164, Des, 2020.
- [6] J. C. Mestika, S. M. Zahra, W. M. Sidik, dan P. Rosyani, "Penerapan Metode TOPSIS Dalam Penentuan E-Commerce Terbaik Untuk Pengambilan Keputusan Yang Efektif," *Jurnal AI dan SPK*, vol. 1, no. 2, hal. 172–177, Sep. 2023.
- [7] A. Mubarak, H. D. Suherman, Y. Ramdhani, dan S. Topiq, "Sistem Pendukung Keputusan Kelayakan Pemberian Kredit Dengan Metode TOPSIS," *Jurnal Informatika*, vol. 6, no. 1, hal. 37–46, Apr. 2019.
- [8] M. S. Lauryn, M. Ibrohim, dan A. Fasambi, "Penerapan Metode TOPSIS Dalam Penentuan Penerima Dana Bantuan Masyarakat Usaha Mikro Kecil Menengah," *Jurnal ProTekInfo*, vol. 10, no. 1, hal. 1–5, Feb. 2023.
- [9] F. R. Darmawan, E. L. Amalia, dan U. D. Rosiani, "Penerapan Metode Topsis pada Sistem Pendukung Keputusan untuk Kota yang Menerapkan Pembatasan Sosial Berskala Besar yang di Sebabkan Wabah Corona," *JUSTIN (Jurnal Sistem dan Teknologi Informasi)*, vol. 9, no. 2, hal. 250–256, Apr. 2021.
- [10] Zulhamdi dan Subandi, "Penerapan Algoritma TOPSIS Untuk Menentukan Pelanggan Terbaik Pada Toko Rangkuti," dalam *Seminar Nasional Mahasiswa Fakultas Teknologi Informasi (SENAFTI)*, Jakarta, Sep. 2022, hal. 594–599.